

1. Write a program to add two numbers 05H and 07H

source code :-

```
ORG 0000H
```

```
MOV A, #05H
```

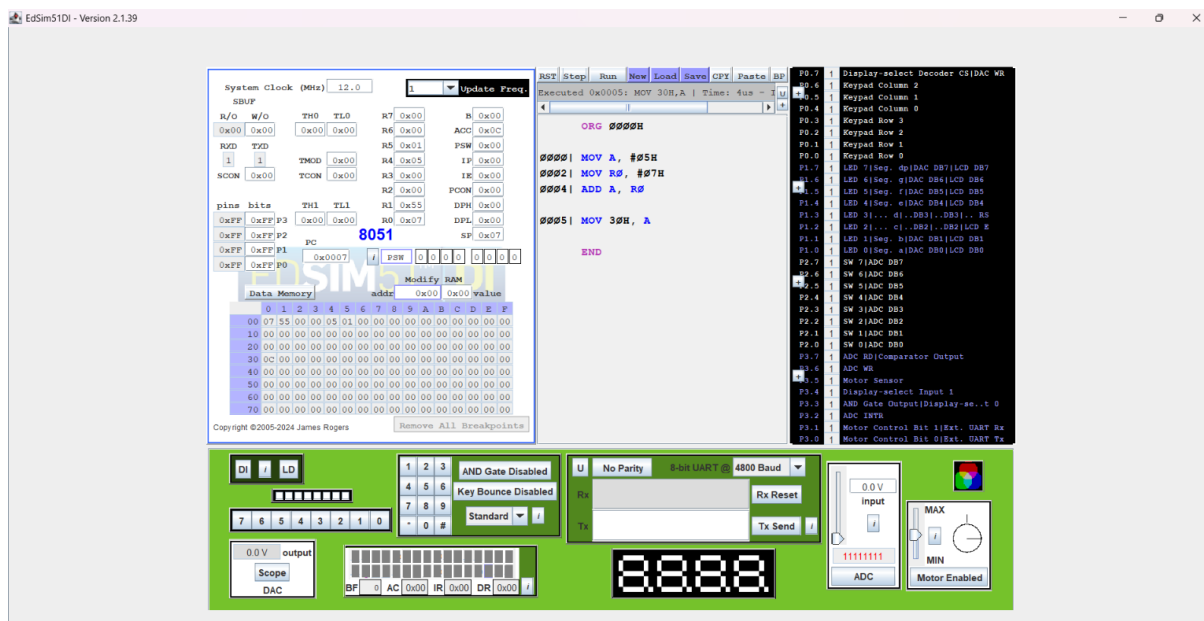
```
MOV R0, #07H
```

```
ADD A, R0
```

```
MOV 30H, A
```

```
END
```

#output



2. Write a program to subtract two numbers 05H and 07H

#source code

```
ORG 0000H
```

```
MOV A, #05H
```

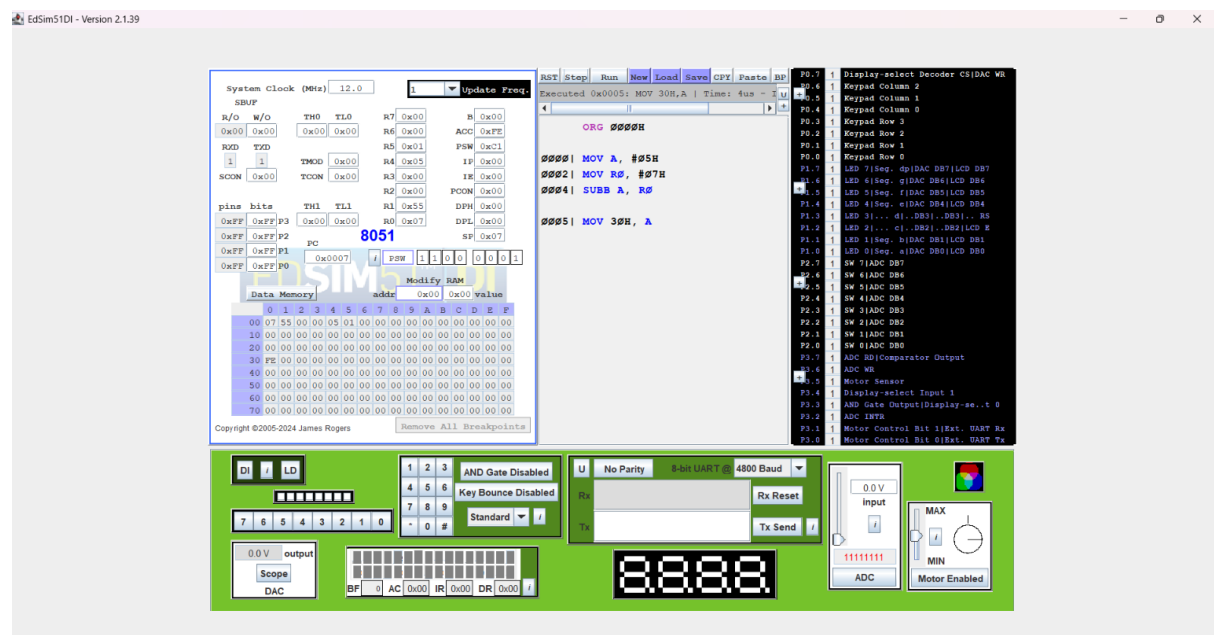
```
MOV R0, #07H
```

```
SUBB A, R0
```

```
MOV 30H, A
```

```
END
```

#output



3. Write a program to multiplication of two number 05H and 07H

#source code

```
ORG 0000H
```

```
MOV A, #05H
```

```
MOV B, #07H
```

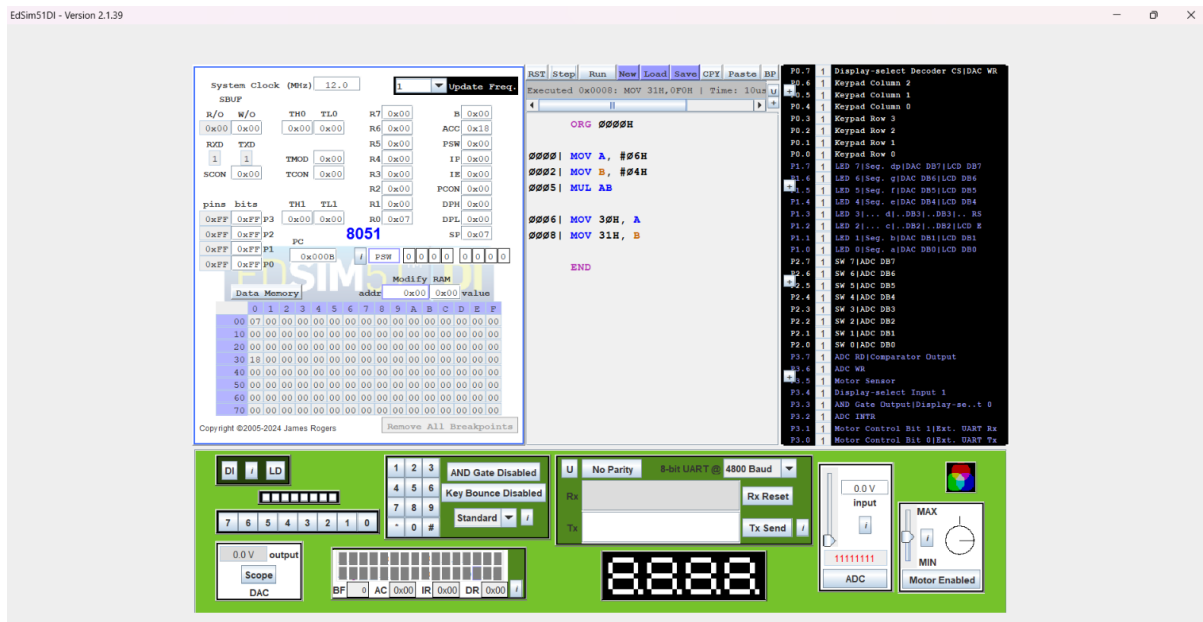
```
MUL AB
```

```
MOV 30H, A
```

```
MOV 31H, B
```

```
END
```

#output



4. Write a program to divide 10H by 03H and store the quotient in R4 and remainder in R5.

#source code

ORG 0000H

MOV A, #10H

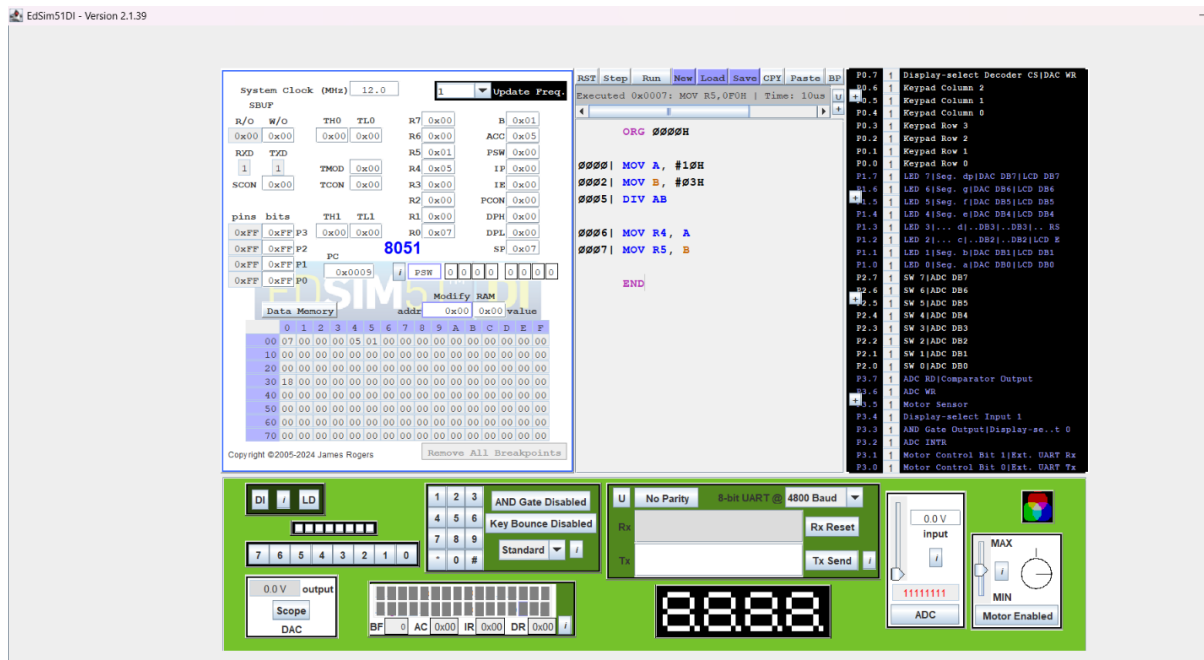
MOV B, #03H

DIV AB

MOV R4, A

MOV R5, B

END



5. Write a program to swap two numbers – 55H IS STORED IN R0 AND AAH IS STORED IN R1 SWAP THE CONTENTS OF R0 AND R1.

#source code

```
ORG 0000H
```

MOV R0, #55H

MOV R1, #0AAH

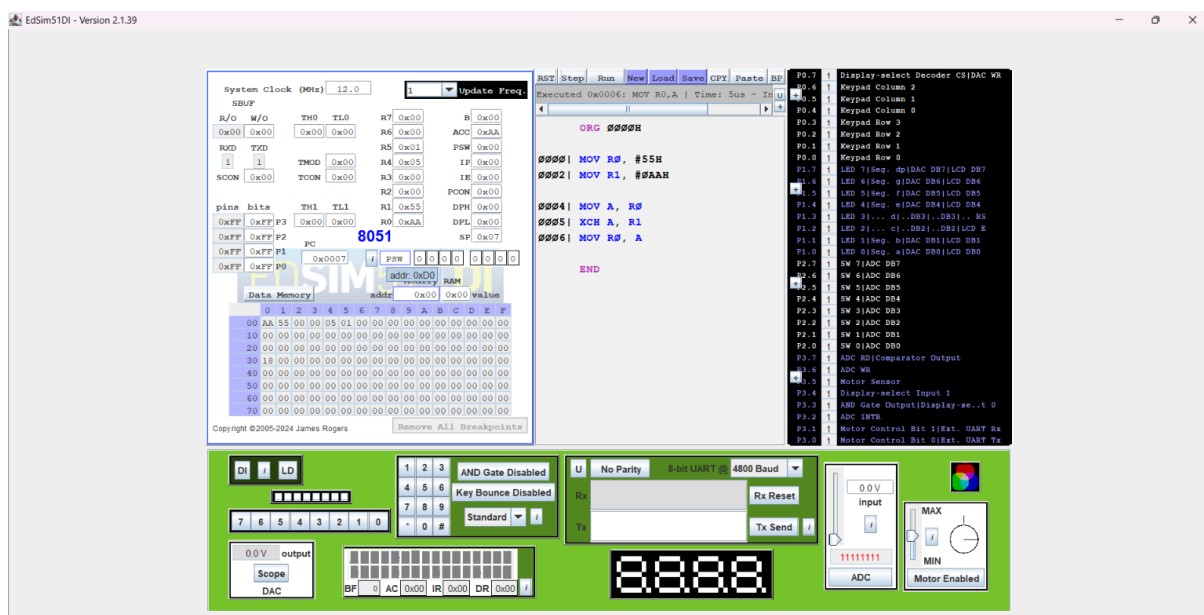
MOV A, R0

XCH A, R1

MOV R0, A

END

#output



6.WRITE A PROGRAM TO BLINK LED ON PORT 1 .

#SOURCE CODE

```
ORG 0000H
```

```
SETB P1.0
```

BLINK:

```
CPL P1.0
```

```
ACALL DELAY
```

```
SJMP BLINK
```

DELAY:

```
MOV R7, #0FFH
```

```
MOV R6, #0FFH
```

```
DJNZ R7, $
```

```
DJNZ R6, $
```

```
RET
```

END

#output

The screenshot displays the EdSim51DI software interface, version 2.1.39. The main window is divided into several sections:

- Assembly Code:** The central pane shows the assembly program for an 8051 microcontroller. It includes the program start at 0000H, setting P1.0, a BLINK loop (CPL P1.0, ACALL DELAY, SJMP BLINK), and a DELAY loop (MOV R7, #0FFH, MOV R6, #0FFH, DJNZ R7, \$, DJNZ R6, \$, RET). The program ends at 8051H.
- Hardware Simulation:** The bottom section shows a simulated hardware environment. It includes a keyboard (DI, LD), a display (8x8 LED matrix), a scope (0.0V output), a DAC (0.0V output), and a motor control section (Motor Sensor, Display-select Input, AND Gate Output, Motor Control Bit, Motor Enabled).
- Registers and Memory:** The top left pane shows the status of registers (R0-R7), memory (RAM, ROM), and the program counter (PC). The PC is currently at 8051H.
- IO Devices:** The right pane shows the status of various I/O devices, including the Display-select Decoder, Keypad, and Motor Control.

7.WRITE A PROGRAM TO BLINK A LED FROM LEFT TO RIGHT

#source code :

ORG 0000H

MOV A,#01H

AGAIN:

MOV P1,A

RL A

SJMP AGAIN

END

#output

iSim51DI - Version 2.1.39

The screenshot displays the iSim51DI software interface, which is used for simulating 8051 microcontroller programs. The main window is divided into several sections:

- Assembly Code Window:** Shows the program code being executed. The code is as follows:

```
ORG 0000H
MOV A,#01H
AGAIN:
MOV P1,A
RL A
SJMP AGAIN
END
```
- Register/Status Window:** Displays the current state of the 8051 registers and status flags. The PC (Program Counter) is highlighted at 0005, and the accumulator (A) contains 01H.
- Memory Window:** Shows the memory contents, with the address 0005 highlighted.
- Hardware Simulation Window:** Displays the simulated hardware components, including the 8051 microcontroller, a 4-bit LCD, a 4-bit DAC, a 4-bit ADC, and a 4-bit motor. The LCD shows the hexadecimal value 01, and the DAC output is 0.0V.
- Output Window:** Shows the output of the simulation, including the LED status (all LEDs are on) and the motor status (Motor Enabled).

The interface also includes a toolbar with buttons for Run, Stop, Pause, and other simulation controls. The status bar at the bottom indicates the current time and instructions executed.

8. Write a program to Display 8051 on 7 seven segment display .

#source code

```
ORG 0000H
```

```
start:
```

```
    SETB P3.3
```

```
    SETB P3.4
```

```
    MOV P1,#10000000B
```

```
    LCALL delay
```

```
    CLR P3.3
```

```
    MOV P1,#11000000B
```

```
    LCALL delay
```

```
    CLR P3.4
```

```
    SETB P3.3
```

```
    MOV P1,#10010010B
```

```
    LCALL delay
```

```
    CLR P3.3
```

```
    MOV P1,#11111001B
```

```
    LCALL delay
```

```
    JMP start
```

```
delay:
```

```
    MOV R0,#300
```

DJNZ R0,\$

RET

END

